Do We Train on Test Data?

The Impact of Near-Duplicates on License Plate Recognition: what the paper says, what our Kaggle run found, and how to use it

Prepared for MD. Rokib Hasan Oli (paper reader and data lead) · Supervisor: Dr. Md Mansur ul Mulk · 18 October 2026

Paper: R. Laroca, V. Estevam, A. S. Britto Jr., R. Minetto, D. Menotti, IJCNN 2023, DOI 10.1109/IJCNN54540.2023.10191584, arXiv 2304.04653v2 (8 pages). Page numbers below are the arXiv PDF pages.
Our run: Kaggle notebook `paper23_kaggle.ipynb`, finished 18 Oct 2026, 65.5 min on Tesla T4 x2.

The whole report on one page

The paper in three lines. The two most used LPR datasets (AOLP, CCPD) put different photos of the *same plate* in both train and test. 47-68 % of AOLP test images and 19 % of CCPD test images have such a "near-duplicate" in training. When the authors re-split the data so no plate is on both sides, six OCR models' error rates more than double on AOLP, CCPD's average accuracy falls from 80.3 % to 77.6 %, and the "best" model changes (p.1-5).

Our run in three lines. We trained the same YOLOv8n plate detector and plate reader twice on the KUET Bangla dataset: once on KUET's own published split (with its Roboflow copies), once on our grouped split. In KUET's published test set, 58.8 % of the plates (114 of 194) are also in training. The plate-reading accuracy a paper would report is **85.57 %**; the fair number is **72.13 %**, so the error almost doubles (14.4 % to 27.9 %, ×1.93), the same effect as in the paper. On the 38 test plates neither model ever saw, the "leaky" model is *not* better (68.4 % vs 71.1 %). Its advantage comes only from plates it had already seen (+12 to +23 points).

Is the training OK? Yes. All 5 jobs ended with `code=0`, all epochs ran, 0 corrupt images, no NaN. There are no errors, but there are 3 things you must know before you quote numbers (section 14): two cross-scores in `results.json` are not valid, the per-condition reading table favours the leaky model for the same reason, and the kit's "relative gap" is not the paper's formula.

What to tell the faculty. "Public Bangla LPR splits leak test plates into training. On KUET this alone makes plate reading look 13 points better than it is. That is why our dataset uses a grouped split, and why our numbers are lower than some published ones: ours are honest." Show 5 things: the problem picture, the paper's table, our leakage audit, our 85.6 → 72.1 result, and the seen/unseen chart (section 17).

GitHub. Upload the ready folder `After Train/github_upload/weights/paper23/` into `weights/` of the CSE498R repo; the steps are in section 21.

How this report is organised

Part	Sections	Read it when
A. Understand the paper	1-9: problem, terms, data, method, results, critique	before you explain the paper to anyone
B. Our experiment	10-14: design, leakage in KUET, results, comparison, health check	before you quote any of our numbers
C. Use it	15-21: research use, benefits, faculty slides, script, Q&A, next steps, GitHub	before the faculty meeting

1. What problem does the paper study?

An automatic license plate recognition (ALPR) system has two steps: **detection** (find the plate in the photo) and **recognition** (read its characters, an OCR task). The paper looks only at recognition, because modern systems crop the plate and **rectify** it (warp it to a flat rectangle) before reading it (p.1).

Researchers compare models on public datasets that are divided into train and test sets. The split is usually by *file*: each image goes to train or to test. But one vehicle is often photographed several times (another second, another angle, another subset). After cropping and rectifying, two such photos of the same plate look almost identical. If one is in train and the other is in test, the model is partly tested on something it has already memorised (p.1, Fig. 1-3).

So the question is: **how much of the reported accuracy is memory and how much is generalisation?** The idea comes from Barz & Denzler (2020), who found the same problem in CIFAR-10/100 ("Do we train on test data? Purging CIFAR of near-duplicates") (p.2).

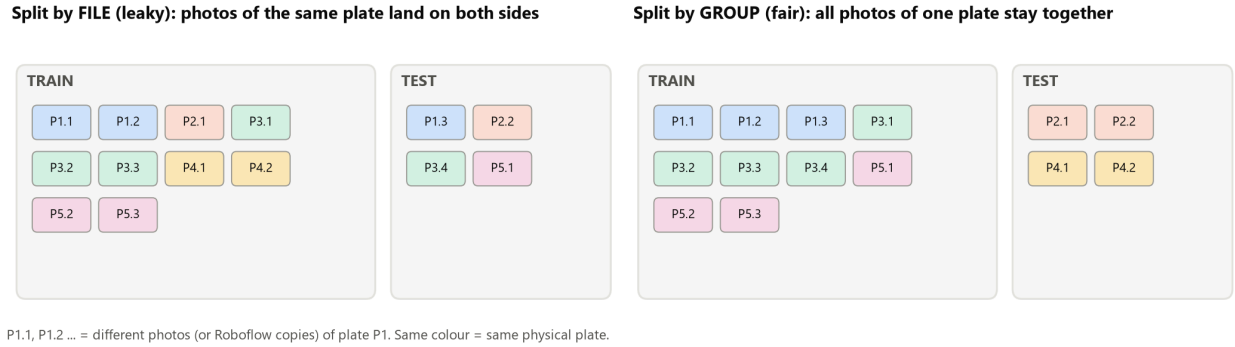


Figure 1 (drawn for this report). Left: a split by file lets photos of one plate land in both train and test. Right: a split by group keeps every photo of a plate on one side. This is the whole idea of the paper.

2. Key terms in plain words

Term	Meaning	Example
Exact duplicate	the same file twice (byte copy) or a re-encoded copy	the same JPEG saved twice
Near-duplicate (this paper)	**different images that show the same license plate** (p.1)	one car photographed in subset AC and again in subset RP
Roboflow copy (our data)	an augmented copy of one photo (noise, brightness, flip) exported as a new file	KUET: about 3 files per photo in train
Data leakage	information from the test set is present during training	test plate also in train
Fair split / duplicate-free split	a split where no test image has a near-duplicate in train	AOLP-Fair-A, CCPD-Fair
Group split	put all images of one group (plate, vehicle, photo cluster) into the same split	our 70/15/15 split, seed 42
Recognition rate	plates read 100 % correctly / all test plates (one wrong character = wrong)	96.94 %
Gap	original accuracy minus fair accuracy, in percentage points (pp)	99.18 - 96.94 = 2.24 pp
Relative gap	gap / original error rate = gap / (100 % - original accuracy) (p.4)	2.24 / 0.82 = 273 %
Memorisation vs generalisation	remembering seen examples vs working on new ones	what a test set must measure

> **Why the relative gap matters.** A drop from 99.18 % to 96.94 % sounds small (2.24 points), but the error goes from 0.82 % to 3.06 %, which is 3.7 times as many wrong plates. The relative gap shows this: 2.24 / 0.82 = 273 %. When accuracy is near 100 %, always look at the error rate.

3. Datasets and how people split them

Dataset	What it is	Split used in the literature	Test images with a near-duplicate in train (p.3, p.5)
AOLP (Taiwan)	2,049 images in 3 subsets: access control AC 681, law enforcement LE 757, road patrol RP 611 (p.2)	**AOLP-A**: random 2:1 train/test. **AOLP-B**: train on AC+LE, test on RP. 20 % of train is validation.	AOLP-A: 320 of 683 = **46.9 %** AOLP-B: 413 of 611 = **67.6 %**
CCPD (mainland China)	366,789 images taken with handheld cameras in one city; subsets Base, Blur, Challenge, DB, FN, Green, Rotate, Tilt, Weather (p.2)	standard split: Base 200k into 100k train + 100k val; all other subsets (about 157k) are test	29,943 of about 157k = **19.1 %**
ReId	182,335 low-resolution plate crops, standard split 105,923 / 76,412 (p.5)	standard split	52,394 of 76,412 = **68.6 %**

Note that AOLP-B, the "harder" protocol that tests on a different subset, has the *most* duplicates: the same cars drive through the access-control gate and past the road-patrol camera.

4. How the authors found the near-duplicates

1. **Step 1, filter by plate text.** Only images whose plates have the same text can be duplicates (p.3).
2. **Step 2, compare pixels.** Among those, they crop and rectify the plates and take the nearest neighbour by Euclidean distance in pixel space. Because step 1 already removed different plates, this simple distance "works reasonably well" (p.3).
3. **Step 3, check by eye.** Fig. 1 shows pairs at the 10th, 50th and 90th percentile of distance. Even the 90th-percentile pair is clearly the same plate.

Their lists of duplicates and the fair splits are public at `raysonlaroca.github.io/supp/lpr-train-on-test/`. There is no training code.

5. How they built the fair splits

They could not replace duplicates with new images (that would change the data distribution), so they re-split while keeping each protocol's character and size (p.3-4):

Fair split	What they changed	What stayed the same	Why this design
AOLP-Fair-A	new random 2:1 split, but all images of one vehicle/plate are kept in the same set	same numbers of train, val and test images	a like-for-like random split, only without leakage
AOLP-Fair-B	**only removed** the duplicates from the test set (RP)	train and val identical to AOLP-B	so nobody can say the drop comes from less training data
CCPD-Fair	Base is still split 100k/100k, but every duplicate goes to **validation**, not train (same for Green)	test set unchanged (the same 157k images)	the test is identical, so any drop is caused by the training data

> **The clever part.** Fair-B and CCPD-Fair each keep one side fixed (train or test). That removes the obvious objection "your fair split is just harder or smaller". Our own experiment copies these two ideas: the "clean part of the published test" is the Fair-B idea, and the seen/unseen check is the controlled version.

6. Models and training

Item	Detail (p.4)
Models (6)	LPR-specific multi-task CNNs: CNNG, Holistic-CNN, Multi-Task. Scene-text models: STAR-Net (CTC), TRBA (attention), ViTSTR-Base (Transformer)
Keras models	Adam, learning rate 1e-3 (ReduceLROnPlateau patience 3, factor 0.1), batch 64, at most 100 epochs, early stop patience 7
PyTorch models	deep-text-recognition-benchmark fork; Adadelta (rho 0.99), 300k iterations, batch 128
Extra data	many synthetic plates: blank templates + random but realistic character sequences + perspective, noise, shadow and colour changes (also applied to real images) (p.4, Fig. 4)
Hardware	AMD Threadripper 1920X, 96 GB RAM, NVIDIA Quadro RTX 8000 (48 GB). Training time not reported

Fairness of comparison	all models trained by the authors with the same protocol in each experiment; no numbers copied from other papers (p.4, footnote 6)

7. Results of the paper

Recognition rate (%) on the original split → the fair split (gap in points; relative gap). Best in each column in bold. Source: Tables I, II, III (p.4-5).

Model	AOLP-A → AOLP-Fair-A	AOLP-B → AOLP-Fair-B	CCPD → CCPD-Fair
CNNG	98.88 → 95.63 (3.25; 290 %)	98.91 → 96.80 (2.11; 194 %)	88.24 → 86.93 (1.31; 11 %)
Holistic-CNN	96.75 → 93.11 (3.64; 112 %)	98.42 → 96.30 (2.12; 134 %)	77.01 → 75.41 (1.60; 7 %)
Multi-Task	97.33 → 93.79 (3.54; 133 %)	98.42 → 95.29 (3.13; 198 %)	83.01 → 81.84 (1.17; 7 %)
STAR-Net	98.69 → 95.83 (2.86; 218 %)	98.47 → 96.46 (2.01; 131 %)	78.53 → 73.33 (**5.20**; 24 %)
TRBA	**99.18** → **96.94** (2.24; 273 %)	98.75 → **97.47** (1.28; 102 %)	75.83 → 71.48 (4.35; 18 %)
ViTSTR-Base	98.74 → **96.94** (1.80; 143 %)	98.75 → 97.31 (1.44; 115 %)	79.06 → 76.37 (2.69; 13 %)

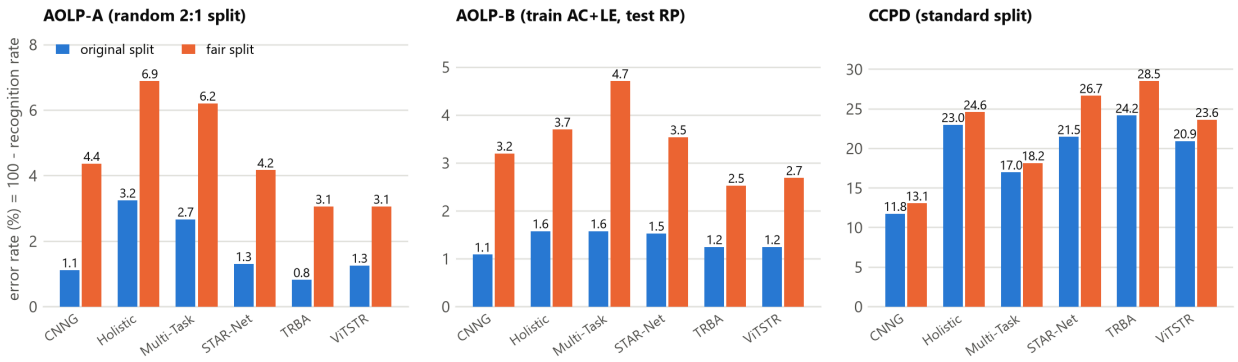


Figure 2 (re-plotted from Tables I-III). Error rate = 100 % - recognition rate. On AOLP the orange bars (fair split) are 2-4 times as high as the blue ones (original split).

The four findings to remember

- **Errors more than double on AOLP.** Every model, both protocols. Relative gaps 102-290 % (p.4).
- **The ranking changes.** On AOLP-B, CNNG is 1st on the original split but only 3rd on the fair split. On AOLP-A, ViTSTR moves from 3rd to tied 1st (p.4-5). So a "new best model" claimed on a leaky split may only be better at memorising.
- **CCPD: smaller relative effect, big absolute effect.** Average 80.3 % → 77.6 %. STAR-Net loses 5.20 points and TRBA 4.35. Even the smallest gap (1.17 pp) is 1,800+ extra misread plates, and 5.2 pp is 8,000+ (p.5). The worst subset is Blur: TRBA 50.2 % → 38.7 % (Table IV).
- **Bigger models may hide it longer.** The ability to memorise grows with model capacity, so the problem may get worse as models get deeper (p.5).

Section IV: the problem is everywhere

- EnglishLP, Medialab LPR and PKU have no official split; people split randomly and get near-duplicates without noticing (p.5, Fig. 5).
- ReId: 68.6 % of test images have a near-duplicate in train, and works using it never mention it (p.5).
- **Across datasets:** CLPD (used as a "cross-dataset" test set) shares vehicles with ChineseLP. Training on one and testing on the other is then not really cross-dataset (p.6, Fig. 7).
- Good example: RodoSol-ALPR was split with no duplicates by design (p.6).

8. Critical reading: strengths and limits

Strengths

- Simple, important question; clear evidence on the two most used datasets and six model families.
- Careful control in AOLP-Fair-B and CCPD-Fair (one side kept fixed), so the drop cannot be blamed on less data.

- All models trained by the authors with one protocol, so the comparison is consistent.
- Duplicate lists and fair splits are public, so anyone can check or reuse them.
- Peer-reviewed (IJCNN 2023, IEEE), short and easy to cite.

Limits (good points to raise yourself before the faculty does)

- **Recognition only.** Detection is not tested. The authors argue full images vary more, but they give no numbers. (Our run adds detection.)
- **Duplicates need plate text.** Step 1 uses the plate text, so unlabelled images and duplicates of *different* plates of the same car are not covered.
- **AOLP-Fair-A changes which images are trained on**, so part of its drop may come from a different (maybe harder) test set. Fair-B controls for this, and Fair-B still shows big gaps.
- **One run per model.** No seeds, no confidence intervals. Small gaps (for example 1.28 pp on 611 images) could partly be noise.
- **No training code**, only the splits. CCPD corner labels are imprecise (p.2, footnote 2), which affects rectification.
- Synthetic data was added for all models. It reduces memorisation, so without it the gaps might be even larger. This is not tested.

9. Check yourself (2 minutes, without the paper)

> In AOLP and CCPD, many test plates also appear (as other photos) in training: 47-68 % of AOLP test images and 19 % of CCPD's. Re-splitting so that no plate is on both sides makes six OCR models' error rates more than double on AOLP and drops CCPD's average from 80.3 % to 77.6 %. The "best" model even changes. Lesson: split by plate or photo group, never by file, or the numbers are inflated.
>
> Two weak points: OCR only (no detection), and duplicates are found through plate text (unlabelled images are not covered). What we take from it: our grouped split, our leakage audit, and the leaky-vs-fair experiment on Bangla data.

10. What we ran

We used the paper's idea on **KUET** (Bangladeshi License Plate Recognition Dataset, Hossain & Hassan; the dataset of paper #1 in our list). KUET has a plate-localisation part and a character-recognition part, each with its own published train/valid/test split exported from Roboflow. The same source is trained in two versions, with the same models and settings:

Version	Data	Detection images train / val / test	Reading crops train / val / test
leaky	KUET's own published split with every Roboflow copy (`extras/leaky_kuet/`). This is how papers that use KUET trained.	5,620 / 277 / 553	3,391 / 320 / 194
fair	the same KUET images in our grouped 70/15/15 split, seed 42, one copy per photo, every plate in one split (`fair_kuet_*.txt`)	1,758 / 311 / 540	908 / 200 / 183

Setting	Value
Plate detector	YOLOv8n (3.01 M parameters, 8.2 GFLOPs) from COCO weights; 640 px; 50 epochs; batch 16
Plate reader	YOLOv8n character detector over 101 tokens (10 digits, the word "Metro", 26 series letters, 64 district words); 320 px; 80 epochs; batch 32. A "rows" decoder turns the boxes into the two-row BRTA plate text
Training	SGD, lr0 0.01, seed 42, deterministic, Ultralytics 8.4.170, PyTorch 2.11.0+cu128, Python 3.13.15
Hardware / time	Kaggle 2× Tesla T4. GPU 0: det_leaky 47.3 min → det_fair 12.5 min → scoring 0.8 min. GPU 1: rec_leaky 28.8 min → rec_fair 9.7 min. Wall time 65.5 min
Metrics	detection: mAP50 and mAP50-95; reading: plate exact % (the paper's recognition rate: whole plate correct)
Test sets	**leaky_test_all** = KUET published test (what papers report); **leaky_test_clean** = its part with no near-duplicate in the published train; **leaky_test_dup** = the rest; **fair_test** = KUET in our test split

What counts as a duplicate here

- **Detection:** a test photo that is in the same near-duplicate cluster as a train photo. Clusters come from the dataset build: perceptual hashes (pHash, dHash) checked by normalised cross-correlation, plus Roboflow base names and MD5.
- **Reading:** a test crop whose **plate text** also occurs in train. No KUET reading test crop is the same *photo* as a train crop (audit: 0 of 218), so all of these are "same plate, other photo": exactly the paper's definition.

11. How leaky is KUET's published split?

Measure	KUET detection	KUET reading	Paper, for scale
train files vs distinct content	5,687 published files from only 1,964 distinct photos (about 2.9 copies each); 5,620 used after label checks	3,391 crops (3,356 distinct files) of only **447 plates** (about 7.5 crops per plate)	-
test items with a near-duplicate in train	79 of 553 = **14.3 %**	114 of 194 = **58.8 %**	AOLP-A 46.9 %, AOLP-B 67.6 %, CCPD 19.1 %

So KUET's reading test set is as leaky as AOLP: more than half of its test plates were already seen in training. The same audit (`dataset/docs/leakage_audit.md`) finds that BLPD (paper #2) is worse for detection: 94 of 213 test photos (44.1 %) have a copy in train, and 270 of 721 reading crops (37.4 %).

12. Results

12.1 Detection: memory shows only at strict IoU

Detector (mAP50-95 %)	published test, all (553)	clean part (474)	duplicated part (79)	our fair test (540)
det_leaky	76.86	76.04	**83.06**	77.45*
det_fair	74.45	75.93	68.24	74.68
difference	+2.41	+0.11	**+14.82**	-

* det_leaky trained on 20 of these 540 photos. det_fair trained on none of KUET's published test photos, so the first three columns are a clean comparison. mAP50 is 97.1-99.5 % in every cell.

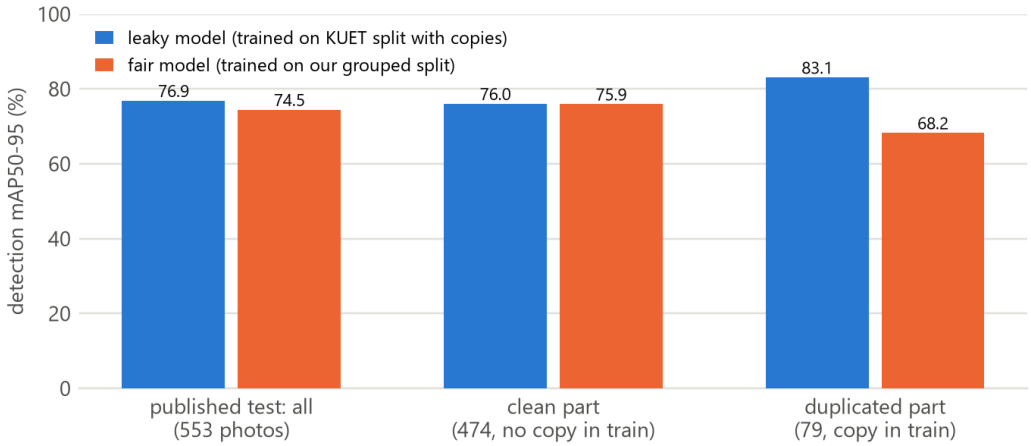


Figure 3. On the 474 clean test photos both detectors are equal. On the 79 photos that have a copy in the leaky training set, the leaky detector is 14.8 points better. That is memory, not skill.

How to read it. At IoU 0.5 (mAP50) plate detection is solved on KUET (about 99 %), so leakage cannot show there. The paper's view that detection is less affected is right for the headline number: the overall gap is only 0.8-2.4 points. But at strict box precision (mAP50-95), on the photos it has seen copies of, the leaky detector is much better. For a model that has never seen them, those 79 photos are *harder* than average (68.2 vs 75.9), yet the leaky model scores 83.1 on them.

12.2 Plate reading: the paper's effect, on Bangla plates

What a paper using KUET's split would report

85.57 %

rec_leaky on the published test (194 plates)

The fair number

72.13 %

rec_fair on our grouped test (183 plates)

The gap is 13.44 points. The error rate goes from 14.43 % to 27.87 %, ×1.93. Relative gap (paper formula) = 13.44 / 14.43 = **93 %**. The paper's AOLP result was "more than twice as high"; ours is almost twice.

Same model (rec_leaky), different parts of the published test	plates	plate exact %
all (what is reported)	194	85.57
plate also in train (near-duplicates)	114	90.35
clean: plate not in train	80	78.75
gap all - clean (the AOLP-Fair-B idea: same model, duplicates removed from test)		6.82 pp; relative gap 6.82 / 14.43 = 47 %

12.3 The controlled check: is the leaky model better, or does it remember?

The two numbers above use different test sets. To compare the readers on *the same* plates, we took the 183 KUET plates of our fair test (the fair reader never trained on any of them) and split them by what the **leaky** reader saw in training. This uses only the saved predictions (`ocr_preds_test.csv`); no model was re-run. The result is in `ocr_overlap_check.json`.

Group of KUET test plates	plates	leaky reader	fair reader	difference
A: the identical crop was in the leaky training set	104	94.23	71.15	**+23.08**
B: same plate, another photo in the leaky training set	41	87.80	75.61	**+12.20**
C: plate in *neither* training set	38	68.42	71.05	**-2.63**

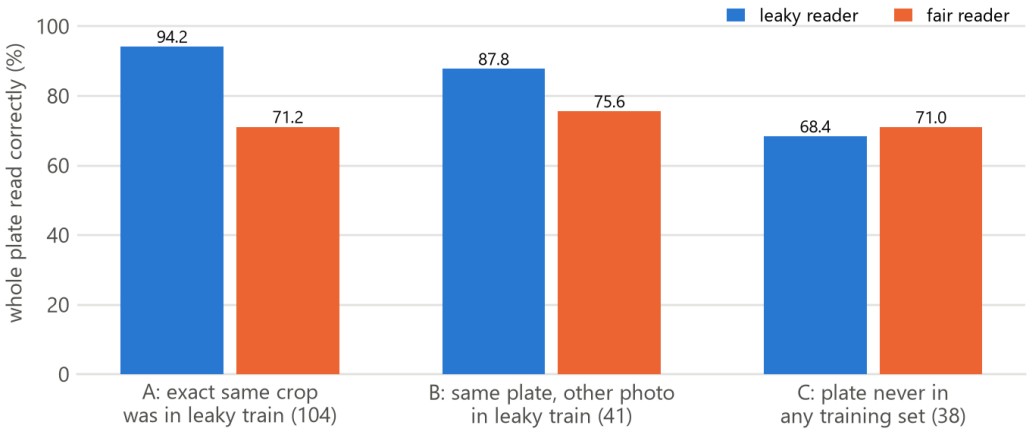


Figure 4. On plates it has never seen (group C) the leaky reader is no better than the fair one. Its whole advantage is on plates it saw: +12 points for the same plate in another photo (the paper's near-duplicate), +23 for the identical crop.

> **This is the strongest result of the run.** The paper says leaky splits measure memory. Here we see it directly: same test plates, same metric, and the gap appears exactly where the model had seen the plate before. Group B is the paper's case (same plate, other photo) and still gives +12 points.

Caution: group C has only 38 plates, so its uncertainty is about ±15 points. Say "no sign of an advantage", not "the fair model is 2.6 points better". On the four other sources, which neither reader saw (anpr, blpd, lpdba, nightlpbd: 468 plates), the leaky reader is 48.1 % vs 46.2 %. That is a small real difference, which fits its 3.7× more training updates.

13. Our results next to the paper

	Paper (AOLP / CCPD)	Ours (KUET, Bangla)
Test items with a near-duplicate in train	AOLP-A 46.9 %, AOLP-B 67.6 %, CCPD 19.1 % (p.3)	reading 58.8 %; detection 14.3 %
Accuracy on the leaky protocol	e.g. TRBA AOLP-A 99.18 %	85.57 %
Accuracy on the fair protocol	TRBA AOLP-Fair-A 96.94 %	72.13 %
Gap, fair protocol (pp)	AOLP 1.28-3.64; CCPD 1.17-5.20	13.44
Relative gap (gap / original error)	AOLP 102-290 %; CCPD 7-24 %	93 %
Same model, duplicates removed from test	AOLP-Fair-B: 1.28-3.13 pp (102-198 %)	6.82 pp (47 %)
Model ranking changes?	yes on AOLP	not tested (one reader architecture)
Detection	not studied	headline +0.8-2.4 mAP50-95; on duplicated photos +14.8
Runs / seeds	one run per model	one run, seed 42

The relative gaps differ because they depend on the starting error: AOLP models start near 1 % error, so a few points of drop is a large multiple. Our reader starts at 14 % error, like CCPD. The **absolute** gap (13.4 points) is bigger than anything in the paper: KUET's reading split repeats each plate about 7.5 times in training, so there is

a lot to memorise.

> **Do not compare 85.57 % with the paper's 99.18 %.** Different country, plate format, data and models. Only compare the *direction and size of the gap*. The kit's table says the same.

14. Is my training OK? Health check of the Kaggle run

> **Yes, the run is correct and complete.**
>
> - All five jobs ended with `code=0`: rec_leaky 29.8 min, rec_fair 10.7 min, det_leaky 47.8 min, det_fair 14.5 min, leaky_vs_fair 0.8 min.
> - Full mode (FAST = False): 50 of 50 detector epochs and 80 of 80 reader epochs ran; no time cap was hit.
> - Every dataset scan reports **0 corrupt** images; no NaN losses; losses fall smoothly.
> - Both zips were packed: `paper23_outputs.zip` 50.9 MB (with weights) and `paper23_results.zip` 3.1 MB (no weights).
> - The only warnings are harmless: Ray `get_trial_id` UserWarning, the debugger "frozen modules" note, and two nbconvert SyntaxWarnings at the very end. Ignore them.

Things to know before you quote numbers (not crashes, but they change the meaning)

#	Issue	Effect	What to do
1	**Two cross-scores are not valid.** The grouped split for the reading crops does not keep KUET's published test crops in test. So rec_fair trained on 135 of the 194 published-test crops, and rec_leaky trained on 104 of the 183 fair-test crops.	rec_fair on the published test (91.24 %) and rec_leaky on the fair test (87.43 %) are inflated. Detection is fine: det_fair saw 0 published test photos.	Do not report these two. Use the controlled check (12.3). Fix the kit later (section 20).
2	**The per-condition reading table favours the leaky reader.** 104 of the 183 KUET crops in our OCR test were in its training data; 81 of the 112 "blurry" plates are KUET.	leaky "blurry" 75.89 % vs fair 59.82 % is mostly memory.	Quote only rec_fair's condition numbers, or the non-KUET sources.
3	**"Relative gap" in the kit is not the paper's formula.** The kit divides by the clean accuracy (8.66 %); the paper divides by the original error rate.	8.66 % looks tiny next to the paper's 112-290 %.	Use 47 % (all vs clean) and 93 % (leaky vs fair protocol).
4	**The fair models got about 3× fewer updates** (the same epochs on 3-4× fewer files). det_fair's best epoch was the last one (50), so it was still improving.	Slightly favours the leaky models, yet they were not better on unseen data, so the conclusion gets *stronger*.	Rerun with equal steps (e.g. det_fair about 160 epochs) for the final report.
5	**rec_leaky memorises:** its validation mAP50-95 stopped improving at about 0.71-0.72 after epoch 20, while its training box loss kept falling (0.77 → 0.36).	Expected with about 7.5 crops per plate.	A good figure for the "memorisation" story (results.csv).
6	**xsort decoder = 0 %** in results.json.	Expected: Bangla plates have two rows, so sorting all characters left to right mixes the rows.	Ignore it; the "rows" decoder is the real one.
7	**Whole OCR test only 53-59 %.**	Both readers trained on KUET only and are tested on 5 sources (different fonts and cameras).	Not a bug; this is a domain-shift result.
8	**One seed, small subsets** (38 and 79 items).	About ±10-15 points of uncertainty on the small groups.	Say "consistent with"; add 3 seeds later.

15. How to use this paper in your research

Use	What to do concretely
1. Justify our dataset design	In the report's Method section, cite Laroca et al. (2023) and our numbers to explain why the BanglaLPR All-in-One split is grouped (near-duplicate clusters + plate text + crops of one photo = one group). Sample sentence: *"Following Laroca et al. [x], who showed that near-duplicates between training and test sets inflate LPR accuracy, we split by group. On KUET's published split, 58.8 % of test plates also occur in training, and plate-reading accuracy falls from 85.6 % to 72.1 % when they are removed."*
2. Read published Bangla numbers carefully	Papers that train and test on KUET's or BLPD's own split (for example #1 and #2 in our list) report numbers that include this leakage. When we compare with them, write "on the published (leaky) split" and give our fair number next to it.
3. A reporting rule for the team	Never re-split the dataset randomly. Report on our fair test. For any OCR model also report **seen vs unseen plates** (as in 12.3). This costs nothing: it uses saved predictions.
4. Make the leakage audit a result	`dataset/docs/leakage_audit.md` covers 14 public Bangla splits (KUET 14.3 % / 58.8 %, BLPD 44.1 % / 37.4 %, NightLPBD 20.7 % / 27.6 %, Zenodo 22.1 %). Before calling it new, search Google Scholar for a similar audit of Bangla datasets. If you find none, it can be a section of the thesis or a short paper: "near-duplicates in Bangladeshi LPR datasets".
5. Choose models fairly	The paper shows the ranking can change. Compare all our models (YOLO char reader, CRNN, PARSeq, TrOCR...) only on the grouped test, or a "better" model may just memorise better.
6. Check across datasets too	Section IV warns about duplicates across datasets. Our build de-duplicated all 12 sources together, so a plate copied between two public sets is one group. Say this when you present cross-source results.

16. What you gain from it

- **Credible numbers.** Faculty and reviewers can trust that our accuracy is not memory.
- **A defence, not a weakness.** When someone asks "why is your accuracy lower than paper X?", you have a peer-reviewed reason and our own measurement.
- **Our own finding on Bangla data,** not only a reproduction. The 85.6 → 72.1 result and the seen/unseen chart are new evidence.
- **Fair model choice.** Decisions based on the grouped test pick the model that generalises.
- **Saves time.** We will not chase a 95 %+ published number that was partly memory.
- **A reusable tool and rule** for the whole team (grouped split, leakage audit, seen/unseen breakdown).

17. What to show the faculty (8 slides)

#	Slide	Content (all files are in this folder or the dataset)
1	The claim	One sentence: "Public Bangla LPR splits leak test plates into training; on KUET this makes plate reading look 13 points better than it is."
2	The problem	Figure 1 of this report (split by file vs by group).
3	The paper	Laroca et al., IJCNN 2023: duplicate fractions (46.9 / 67.6 / 19.1 %) + Figure 2 (errors double).
4	Our audit	Table from leakage_audit.md: KUET, BLPD, NightLPBD, Zenodo duplicate rates.
5	Our experiment	Leaky vs fair design (section 10 tables): same models, same settings, two splits.
6	Headline result	85.57 % → 72.13 %; error ×1.93 (section 12.2).
7	Proof it is memory	Figure 4 (groups A/B/C) and Figure 3 (detection: equal on clean photos, +14.8 on duplicated photos).
8	So what + next	Our grouped split; our numbers are honest; limits (one seed, 38 plates); next steps (section 20).

18. What to explain: a 2-minute script

> "Paper #23 is by Laroca and colleagues, IJCNN 2023. They noticed that the most used license-plate datasets put different photos of the *same plate* in both train and test. After cropping, these photos look

> almost the same, so the model is partly tested on what it memorised. In AOLP almost half to two-thirds of the test images have such a near-duplicate. When they re-split so that no plate is on both sides, error rates of six OCR models more than doubled, and the best model changed.
>
> We checked our Bangla data the same way. In KUET's published split, 59 % of the test plates also appear in training, because Roboflow exports several copies of each photo and the same cars appear many times. We trained the same YOLOv8n models on KUET's split and on our grouped split. Plate reading looks like 85.6 % on the leaky split but is 72.1 % on the fair one: the error almost doubles, as in the paper. On plates that neither model had ever seen, the leaky model is not better at all. Its advantage is only on plates it had already seen.
>
> That is why our dataset is split by group, and why our numbers can be lower than some published Bangla results: ours measure generalisation."

19. Questions the faculty may ask, with answers

Q1. What exactly is a near-duplicate?
Different images showing the same physical plate (the paper's definition). In our data also Roboflow-augmented copies of one photo. We find them with perceptual hashes checked by cross-correlation, Roboflow base names, MD5, and the plate text.

Q2. Is the fair model worse just because it has fewer training images?
No. The distinct content is similar (447 plates in leaky train vs about 422 in fair train); the leaky set mainly has more *copies*. The leaky models even got about 3× more updates. Still, on plates neither model saw (group C) the leaky reader is not better, and on clean detection photos both detectors are equal (76.0 vs 75.9).

Q3. 38 plates is small. Is that significant?
Not on its own; the uncertainty is about ±15 points. The claim is "no sign of an advantage on unseen plates". The large, consistent gaps in groups A (+23) and B (+12) are the main evidence. Next: 3 seeds and the BLPD dataset.

Q4. Does it matter for detection?
Less for the headline. mAP50 is about 99 % either way, which matches the paper's remark that full images vary more. At strict IoU (mAP50-95), on duplicated photos the leaky detector is +14.8 points, so memorisation exists there too.

Q5. Why not just delete duplicates from the test set instead of re-splitting?
We did both. "Clean part of the published test" is the paper's AOLP-Fair-B idea (85.6 → 78.8 for the same model). Re-splitting is the Fair-A idea. Both show the same direction.

Q6. Why is your accuracy lower than published Bangla papers?
Many published results use the leaky published splits. Our test is grouped and covers five sources. On KUET alone, leakage is worth 13 points of plate reading.

Q7. How did you find duplicates in images that have no plate text?
The dataset build compares perceptual hashes (pHash, dHash) and verifies candidate pairs with normalised cross-correlation. It also links Roboflow copies by their base file name and byte copies by MD5. 51,348 images from 12 sources were checked together.

Q8. Doesn't removing copies throw away useful augmentation?
We remove only duplicate *files*. Augmentation is done online during training (YOLO mosaic, HSV, flips), so the model still sees variations. It just does not see them from the test set.

Q9. What would you change in the experiment?
Equal training steps for both versions, 3 seeds, keep KUET's published test crops in test for the reading split too, add a stronger reader (PARSeq, paper #31), and repeat on BLPD.

20. Next steps (in order)

1. **Upload the results** (section 21) and fill the team table: put `paper23_outputs.zip` in `docs/results/` and run `python add_results.py paper23_outputs.zip --member "Rokib Hasan Oli"`.
2. **Fix the kit** (`docs/_kit`): keep KUET's published test crops in the fair reading test (or score cross-models only on unseen plates); add the paper-style relative gap; add the A/B/C seen/unseen table to `p23_leaky.py score`. Then rebuild the notebook with `build_kits.py`.

3. **Rerun with equal steps** (fair models about 3× the epochs) and 3 seeds; report mean ± std.
4. **Repeat on BLPD** (44.1 % of detection test photos leaky), the dataset of paper #2.
5. **Stronger reader:** run leaky vs fair with PARSeq (kit #31). The paper expects higher-capacity models to memorise more.
6. **Write it up:** one paragraph + Figure 4 in the Method/Dataset chapter; the audit table in the Dataset chapter.

21. Uploading to GitHub (and Drive)

Everything is prepared in `After Train/github_upload/weights/paper23/MD.Rokib.Hasan.Oli/`, in the same layout as Md. Yousuf's `weights/paper03/Md.Yousuf/`:

File	Size	Why
README.md	-	what the package is, main results, limits (like your teammate's README)
best_det_leaky.pt, best_det_fair.pt	5.9 MiB each	the two plate detectors
best_rec_leaky.pt, best_rec_fair.pt	6.6 MiB each	the two plate readers
paper23_results.zip	3.0 MiB	all results, logs, curves and code, without weights
results_vs_paper.md	2 KB	the kit's ours-vs-paper table
ocr_overlap_check.json	1 KB	the controlled seen/unseen check (section 12.3)
Paper23_Study_Report.pdf (optional)	-	this report, if you want it in the repo

Not to GitHub: `paper23_outputs.zip` (48.5 MiB, includes last.pt) goes to the team Drive folder `498R / weights / paper23 /`. It is ready in `After Train/for_drive/`. Never upload `results.zip` (174 MB, the whole Kaggle output).

Way 1: in the browser (easiest)

1. Open github.com/Rokib-Hasan-Oli/CSE498R and click the `weights` folder.
2. Click **Add file** → **Upload files**.
3. From File Explorer, drag the whole folder `After Train\github_upload\weights\paper23` into the page (Chrome/Edge keep the folder structure). Check that the list shows `paper23/MD.Rokib.Hasan.Oli/...` with 8 files (9 with the PDF).
4. Commit message: `Add paper #23 results: leaky vs fair KUET split (Rokib)`. Optional description: "YOLOv8n detector + char reader; reading 85.57 % leaky vs 72.13 % fair; see README".
5. Choose **Commit directly to the main branch** → **Commit changes**. Open the folder on GitHub and check that the README is shown.

Way 2: with git (Git is installed on this PC)

```
cd D:\498R
git clone https://github.com/Rokib-Hasan-Oli/CSE498R.git
   (now copy the folder After Train\github_upload\weights\paper23
    into D:\498R\CSE498R\weights\ with File Explorer)
cd CSE498R
git add weights/paper23
git commit -m "Add paper #23 results: leaky vs fair KUET split (Rokib)"
git push
```

The first push asks you to sign in to GitHub in the browser. Files under 100 MB are fine with git (the web upload limit is 25 MB per file).

Appendix: where every number comes from

Number	Source file
paper numbers	arXiv 2304.04653v2, pages given in the text (Tables I-IV on p.4-5)
leaky/fair scores, gaps	`results.zip → runs/leaky_vs_fair/results.json`
training settings, times, curves	`runs/<run>/results.json, args.yaml, results.csv`; Kaggle log

seen/unseen groups A/B/C	`ocr_overlap_check.json` (from `runs/rec_*/ocr_preds_test.csv` + KUET labels)
overlap of the two splits (20, 104, 135 crops)	file-hash comparison of `extras/leaky_kuet/*/images` and `*/fair_kuet_*.txt`
KUET/BLPD leakage, copies per photo	`dataset/docs/leakage_audit.md`, `dedupe_report.md`

```
@inproceedings{laroca2023do,
  title     = {Do We Train on Test Data? {T}he Impact of Near-Duplicates on License Plate Recognition},
  author    = {Laroca, Rayson and Estevam, Valter and Britto Jr., Alceu S.
               and Minetto, Rodrigo and Menotti, David},
  booktitle = {International Joint Conference on Neural Networks (IJCNN)},
  pages     = {1--8},
  year      = {2023},
  doi       = {10.1109/IJCNN54540.2023.10191584}
}
```